

Two-level random feature methods for elliptic partial differential equations over complex domains

Yifei Sun ^a, Jingrun Chen ^{b,*}

^a School of Mathematical Sciences, Soochow University, Jiangsu, 215006, China

^b School of Mathematical Sciences and Suzhou Institute for Advanced Research, University of Science and Technology of China, Jiangsu, 215217, China

ARTICLE INFO

MSC:

Primary 65N35
65N55
65Y20

Keywords:

Random feature method
Domain decomposition method
Partial differential equation
Least-squares problem

ABSTRACT

Solving partial differential equations (PDEs) is widely used in scientific and engineering applications. Challenging scenarios include problems with complicated solutions and/or over complex domains. To solve these issues, tailored approximation space and specially designed meshes are introduced in traditional numerical methods, both of which require significant human efforts and computational costs. Recent developments in machine learning-based methods, especially the random feature method (RFM), remove the usage of meshes and thus can be easily applied to problems over complex domains. However, as the solution and/or geometry complexity increases, a significant number of collocation points and random feature functions are needed, which results in a large-scale linear problem that is difficult to solve. In this work, by combining the idea of domain decomposition and RFM, we propose two-level RFMs to solve elliptic PDEs. First, complex domains are decomposed by dividing their bounding box along the coordinate planes, and the resulting smaller problems over subdomains are solved using local random features. Only the output-layer weights of the neural network, which serve as a compressed representation of the complicated solution, are communicated between adjacent subdomains, making this step highly parallelizable. Second, a one-time QR decomposition is applied for local problems at the fine level and one global problem at the coarse level and is reused repeatedly in the iterative process. Moderate numbers of iterations are needed to achieve a global convergence. Therefore, our method reduces the computational cost significantly without sacrificing accuracy. Three-dimensional elliptic problems with complicated solutions and/or over complex domains, including the Poisson equation, multiscale elliptic equation, and elasticity problems, are used to demonstrate the efficiency and robustness of our method. For the same accuracy requirement, our method solves these problems within a timescale of 100 s, while traditional methods typically take longer for the whole process or cannot even get a solution due to the difficulty of generating a mesh.

1. Introduction

Complex domains are ubiquitous in the science and engineering spectrum, particularly notable in mechanical engineering where components frequently display complex and irregular geometries. Solving partial differential equations (PDEs) over such domains and/or with complicated solutions is a well-known challenge. Numerical methods, such as finite difference method [1], finite element

* Corresponding author.

E-mail addresses: yfsun99@stu.suda.edu.cn (Y. Sun), jingrunchen@ustc.edu.cn (J. Chen).

<https://doi.org/10.1016/j.cma.2025.117961>

Received 23 November 2024; Received in revised form 22 March 2025; Accepted 23 March 2025

Available online 8 April 2025

0045-7825/© 2025 Elsevier B.V. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

method (FEM) [2], spectral method [3] among others, have been developed to solve a wide range of PDEs for several decades. To solve these issues, tailored approximation space and specially designed meshes are introduced in traditional numerical methods, both of which require significant human efforts and computational costs [4–12].

In recent years, deep neural networks have been utilized to solve PDEs [13–18]. One of the most notable successes of using deep learning is the ability to solve high-dimensional problems, which traditional algorithms have been unable to handle. For low-dimensional problems, efforts have been made to integrate the strengths of both traditional and machine learning-based methods [19–22]. The random feature method (RFM) leverages the benefits of machine learning, particularly its capacity to handle complex geometries while achieving accuracy comparable to traditional methods. RFM has shown promise for problems such as linear elasticity and Stokes flow over complex geometries [22] and has developed further for interface and time-dependent problems [23,24].

Domain decomposition methods (DDMs) represent a well-established family of techniques used either as iterative solvers or as preconditioners for solving linear and nonlinear PDEs, particularly suited for complex large-scale problems on parallel computing systems [25–32]. Numerous mature software packages, including PETSc [33] and FreeFEM [34], have implemented DDM to support FEM, even extending to commercial applications [35,36].

The latest several attempts are made to incorporate DDM into machine learning-based methods for solving PDEs [37–39]. In RFM, only parameters in the output layer, i.e. the coefficients of random feature functions, are optimized, which results in a convex optimization system when solving linear PDEs. The optimization in RFM is a least squares problem, an area that remains underdeveloped, particularly for ill-conditioned problems with sparse structures. The software featured by DDM is primarily designed for FEM, lacking support for least-squares problems and relying heavily on mesh generation.

The objective of this work is to propose an efficient solver that leverages DDM to enhance the performance and effectiveness of RFM, specifically addressing large-scale linear systems arising from increased complexity of solution and/or geometry. In two-level RFMs, complex domains are decomposed by dividing their bounding box along the coordinate planes, with the resulting smaller subdomain problems solved using local random features. QR decomposition is performed only once for local problems at the fine level and one global problem at the coarse level and is then reused repeatedly in the iterative process. Only the output-layer weights of the neural network, which serve as a compressed representation of the complicated solution, need to be communicated between adjacent subdomains, making this step highly parallelizable. This approach significantly reduces the problem size to approximately $1/|I|^2$ of the original without any loss in accuracy, where $|I|$ is the number of subdomains.

This article is organized as follows. In Section 2, we will introduce two-level RFM schemes to solve the optimization problem in RFM, including a two-step mixed scheme “DDM-2+1”. Properties and advantages of the scheme “DDM-2+1” will be elaborated in detail in Section 3 and more numerical results with complicated solutions are provided in Section 4. In Section 5, we focus on the complex geometry encountered in structural analysis. These cases include significant challenges for FEM. Conclusions are drawn in Section 6.

2. Methodology

Consider the following elliptic problem

$$\begin{cases} \mathcal{L}u(\mathbf{x}) = f(\mathbf{x}) & \mathbf{x} \in \Omega, \\ \mathcal{B}u(\mathbf{x}) = g(\mathbf{x}) & \mathbf{x} \in \partial\Omega, \end{cases} \quad (1)$$

where Ω is a bounded and connected domain in $\mathbb{R}^d$. This formulation encompasses a variety of classical problems, Poisson equation, multiscale elliptic equation, and elasticity problems, and is widely applied in structural analysis and solid mechanics.

2.1. Random feature method

The features in the RFM come from two-layer neural networks with the inner parameters held fixed and chosen to be random. To be specific, consider $\mathbf{X} = [\mathbf{x}_1, \dots, \mathbf{x}_N] \in \mathbb{R}^{d \times N}$ to be the d -dimensional input vectors, $\mathbf{A} = [a_{i,j}] \in \mathbb{R}^{d \times M}$ and $\mathbf{b} = [b_j] \in \mathbb{R}^M$ to be a random matrix for weight and bias. Given random weight $\mathbf{A}$ and bias $\mathbf{b}$, it is natural to generate a cluster of multi-variable random feature functions

$$\phi_j(\mathbf{x}) := \sigma\left(\sum_{k=1}^d x_k a_{k,j} + b_j\right), \quad (2)$$

where $\mathbf{x} = (x_k) \in \mathbb{R}^d$, $\sigma : \mathbb{R} \rightarrow \mathbb{R}$ is a smooth activation function. Derivatives of random feature functions can be computed by the chain rule, known as automatic differentiation in machine-learning software.

To accommodate local variations in the solution, we introduce local random features and piece them together using partition of unity (PoU). To construct the PoU functions, we start with a set of points indexed by a multi-index $\mathbf{i}$, where $\mathbf{i} = (i_1, \dots, i_d)$ consists of d components. Each of these points serves as the center for a component in the partition. The normalized coordinate is constructed as follows:

$$\tilde{\mathbf{x}} = \frac{1}{r_i}(\mathbf{x} - \mathbf{x}_i), \text{ for } \mathbf{i} \in I, \quad (3)$$

where $\mathbf{r}_i \in \mathbb{R}^d$ is preselected. In terms of the PoU functions, when $d = 1$, we use the indicator function

$$\psi_{i_1}^a(\mathbf{x}) = \mathbb{I}_{-1 \leq \tilde{x} < 1},$$

or

$$\psi_{i_1}^b(\mathbf{x}) = \begin{cases} \frac{1+\sin(2\pi\tilde{x})}{2} & -\frac{5}{4} \leq \tilde{x} < -\frac{3}{4} \\ 1 & -\frac{3}{4} \leq \tilde{x} < \frac{3}{4} \\ \frac{1-\sin(2\pi\tilde{x})}{2} & \frac{3}{4} \leq \tilde{x} < \frac{5}{4} \\ 0 & \text{otherwise} \end{cases}$$

with a smooth intersection. A high-dimensional PoU function can be constructed using the tensor product of one-dimensional PoU functions $\psi_i(\mathbf{x}) = \prod_{k=1}^d \psi_{i_k}(\mathbf{x}_k)$. Note that the PoU functions used have a compact support.

Putting together, the approximate solution u_M is given by

$$u_M(\mathbf{x}) = \sum_{i \in I} \psi_i u_{M,i}(\mathbf{x}) = \sum_{i \in I} \psi_i(\mathbf{x}) \left(\sum_{j=1}^{M_i} u_{i,j} \phi_{i,j}(\mathbf{x}) \right). \quad (4)$$

For vectorial solutions, we approximate each component of the solution individually using (4), i.e.,

$$\mathbf{u}_M(\mathbf{x}) = \left(u_M^1(\mathbf{x}), \dots, u_M^{K_I}(\mathbf{x}) \right)^T,$$

where K_I represents the output dimension. A simple choice of the loss function for (1) is as follows:

$$\text{Loss} = \sum_{\mathbf{x}_i \in \Omega} \|\lambda_{I_i}(\mathcal{L}\mathbf{u}_M(\mathbf{x}_i) - \mathbf{f}(\mathbf{x}_i))\|_{l^2}^2 + \sum_{\mathbf{x}_k \in \partial\Omega} \|\lambda_{Bk}(\mathcal{B}\mathbf{u}_M(\mathbf{x}_k) - \mathbf{g}(\mathbf{x}_k))\|_{l^2}^2. \quad (5)$$

Here $\{\lambda_{I_i}\}$ and $\{\lambda_{Bk}\}$ are the penalty parameters, and $\{\mathbf{x}_i\} \subset \Omega$ and $\{\mathbf{x}_k\} \subset \partial\Omega$ are collocation points. An adaptive rescaling strategy is useful here to determine the values of penalty parameters [22]. Minimization of (5) is a convex optimization problem, which can be reformulated as a linear least-squares problem equivalently. The standard solution for least squares is to use QR decomposition as a direct method or LSQR as an iterative method. However, as the complexity of the solution and/or geometry increases, a substantial number of collocation points and random feature functions become necessary, leading to a large-scale linear system that is challenging to solve by direct and iterative methods.

2.2. Domain decomposition method

DDMs encompass a variety of techniques designed to tackle linear algebra problems, particularly within simulations, by effectively utilizing parallel computing architectures. These methods partition a large domain into smaller subdomains, which can be processed independently and in parallel, significantly enhancing computational efficiency and scalability.

2.2.1. Original Schwarz algorithm

The original Schwarz algorithm, also known as the Gauss–Seidel Schwarz method, is an iterative method based on solving sub-problems alternatively. Let $\Omega \subset \mathbb{R}^d$ be the union of two subdomains Ω_1 and Ω_2 ; see Fig. 1. Consider the following Poisson problem

$$\begin{aligned} -\Delta u &= f \text{ in } \Omega, \\ u &= 0 \text{ on } \partial\Omega. \end{aligned} \quad (6)$$

The problem is solved by updating from (u_1^ℓ, u_2^ℓ) to $(u_1^{\ell+1}, u_2^{\ell+1})$ with the following scheme:

$$\begin{aligned} -\Delta u_1^{\ell+1} &= f \text{ in } \Omega_1, & -\Delta u_2^{\ell+1} &= f \text{ in } \Omega_2, \\ u_1^{\ell+1} &= 0 \text{ on } \partial\Omega_1 \setminus \Gamma_1, & u_2^{\ell+1} &= u_1^{\ell+1} \text{ on } \Gamma_2, \\ u_1^{\ell+1} &= u_2^\ell \text{ on } \Gamma_1, & u_2^{\ell+1} &= 0 \text{ on } \partial\Omega_2 \setminus \Gamma_2. \end{aligned} \quad (7)$$

Schwarz proved the convergence of the algorithm, thereby first establishing the well-posedness of the Poisson problem in complex geometries [40].

2.2.2. Jacobi Schwarz algorithm (DDM-1)

The formulation (7) can be extended to an arbitrary complex geometry with multiple subdomains. A bounding box is defined as the smallest hyperrectangle that can fully enclose a geometry. Given a bounded domain $\Omega \in \mathbb{R}^d$, we can its bounding box, a d -dimensional hyperrectangle V defined as

$$V = \min_{r_k} \{V_{\mathbf{x},r} \mid \Omega \subseteq V_{\mathbf{x},r}\}, \quad (8)$$

where

$$V_{\mathbf{x},r} = \prod_{k=1}^d (x_k - r_k, x_k + r_k). \quad (9)$$

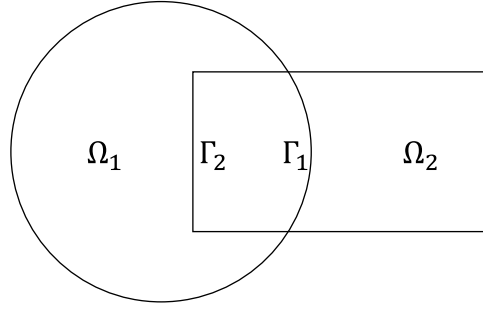


Fig. 1. The domain Ω with the subdomains Ω_1 and Ω_2 .

Its subdomains, also hyperrectangles, are given by:

$$V = \bigcup_{i \in I} V_i := \bigcup_{i \in I} \left(\prod_{k=1}^d (x_{i,k} - r_{i,k}, x_{i,k} + r_{i,k}) \right), \quad (10)$$

where $i = (i_1, \dots, i_d)$ is a multi-index with d components and V_i are overlapping. The pair $(\mathbf{x}_i, \mathbf{r}_i) \in \mathbb{R}^d \times \mathbb{R}^d$ specifies the position and size of each d -dimensional hyperrectangle. Here, $I = \prod_{k=1}^d \{1, \dots, n_k\}$ denotes the Cartesian product of integer sets, defining the indices of subdomains bounded by n_k in each dimension. Each subdomain is constructed by first partitioning the bounding box V uniformly along each dimension into n_k equal parts, and then enlarging each subdomain by adding an overlap of size $\delta_k > 0$ between neighboring subdomains. Specifically, for the k th dimension, let $\Delta_k = 2r_k/n_k$ be the uniform width of each partition, where $[x_k - r_k, x_k + r_k]$ denotes the extent of V in dimension k . The center coordinate $x_{i,k}$ of subdomain V_i is given by

$$x_{i,k} = x_k - r_k + \left(i_k - \frac{1}{2}\right) \Delta_k,$$

and the half-width (radius) is

$$r_{i,k} = \frac{1}{2} (\Delta_k + \delta_k).$$

This construction yields overlapping subdomains, where the intersection between adjacent intervals in each dimension has a fixed length of δ_k .

The boundaries of these hyperrectangles are defined as

$$\gamma_{i,k'}^- := \{x_{k'} - r_{k'}\} \times \prod_{\substack{k=1 \\ k \neq k'}}^d [x_{i,k} - r_{i,k}, x_{i,k} + r_{i,k}] \quad (11)$$

and

$$\gamma_{i,k'}^+ := \{x_{k'} + r_{k'}\} \times \prod_{\substack{k=1 \\ k \neq k'}}^d [x_{i,k} - r_{i,k}, x_{i,k} + r_{i,k}]. \quad (12)$$

Subsequently, we can define the subdomains of a complex geometry as

$$\Omega_i := \Omega \cap V_i \quad (13)$$

with boundaries

$$\partial\Omega_i = (\partial\Omega \cap \bar{V}_i) \cup \left(\bigcup_{k'=1}^d \Gamma_{i,k'}^- \right) \cup \left(\bigcup_{k'=1}^d (\Gamma_{i,k'}^+) \right), \quad (14)$$

where

$$\Gamma_{i,k'}^+ = \gamma_{i,k'}^+ \cap \Omega, \quad (15)$$

and

$$\Gamma_{i,k'}^- = \gamma_{i,k'}^- \cap \Omega. \quad (16)$$

These allow for applying the DDMs; see Fig. 2(a).

In the Jacobi-Schwarz algorithm, sub-problems are solved in parallel across all subdomains. Consider the elliptic problem of the form (1), the Jacobi Schwarz algorithm is given by

$$\begin{cases} \mathcal{L}u_i^{\ell+1} = f & \text{in } \Omega_i, \\ \mathcal{B}u_i^{\ell+1} = g & \text{on } \partial\Omega \cap \bar{V}_i, \\ u_i^{\ell+1} = u_{i-e_{k'}}^{\ell} & \text{on } \Gamma_{i,k'}^-, \text{ for } k' = 1, \dots, d, \\ u_i^{\ell+1} = u_{i+e_{k'}}^{\ell} & \text{on } \Gamma_{i,k'}^+, \text{ for } k' = 1, \dots, d, \end{cases} \quad (17)$$

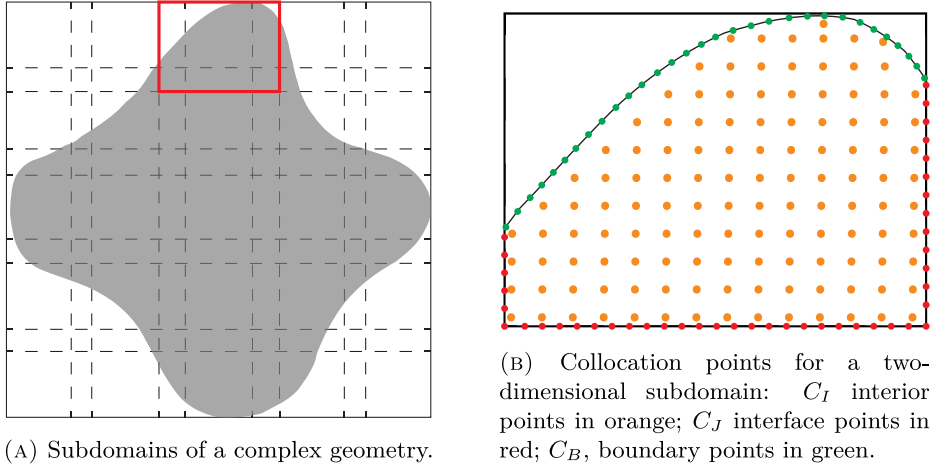


Fig. 2. Subdomains and collocation points of a complex geometry.

where u_i belongs to the space generated by local random features $\{\phi_{i,j}\}$. The loss function for each subdomain is as follows:

$$\begin{aligned} \text{Loss} = & \sum_{\mathbf{x}_i \in C_{I,i}} \left\| \lambda_{I,i} (\mathcal{L} u_{M,i}(\mathbf{x}_i) - f(\mathbf{x}_i)) \right\|_{l_2}^2 \\ & + \sum_{\mathbf{x}_k \in C_{B,i}} \left\| \lambda_{B,k} (B u_{M,i}(\mathbf{x}_k) - g(\mathbf{x}_k)) \right\|_{l_2}^2 \\ & + \sum_{k'=1}^d \sum_{\mathbf{x}_j \in C_{J,i}^-} \left\| \lambda_{J,j}^- (u_{M,i}(\mathbf{x}_j) - u_{i-e_{k'}}(\mathbf{x}_j)) \right\|_{l_2}^2 \\ & + \sum_{k'=1}^d \sum_{\mathbf{x}_j \in C_{J,i}^+} \left\| \lambda_{J,j}^+ (u_{M,i}(\mathbf{x}_j) - u_{i+e_{k'}}(\mathbf{x}_j)) \right\|_{l_2}^2, \end{aligned} \quad (18)$$

where the set of interior points $C_{I,i}$ is sampled in Ω_i , the set of boundary points $C_{B,i}$ is sampled on $\partial\Omega \cap \bar{V}_i$, the sets of interface points $C_{J,i}^-$ and $C_{J,i}^+$ are sampled on $\Gamma_{i,k'}^-$ and $\Gamma_{i,k'}^+$ respectively; see Fig. 2(b) for an illustration.

2.2.3. Two-level Schwarz method (DDM-2)

RFM relaxes the requirement, often seen in kernel methods and other traditional approaches, that the number of basis M must equal the number of conditions N . Following mesh-dependent methods, with a minor variation, we can define coarse and fine levels in RFM. Let $\{\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_N\} \subseteq \bar{\Omega}$ be some scattered points. The fill distance h is usually defined as

$$h = \max_{1 \leq i \leq N} \min_{j \in \{1, \dots, N\} \setminus \{i\}} \|\mathbf{x}_i - \mathbf{x}_j\|_{l_2}. \quad (19)$$

The distinction between coarse and fine levels can be measured by the magnitude of h .

We introduce a global coarse level, whose size is comparable in scale to one of the small sub-problems defined in (17). We denote the solution approximated at this coarse level as $\mathbf{u}^C$. The conditions for $\mathbf{u}^C$ are formulated as follows:

$$\begin{cases} \mathcal{L} \mathbf{u}^C = f & \text{in } \Omega, \\ B \mathbf{u}^C = g & \text{on } \partial\Omega. \end{cases} \quad (20)$$

The solution at the finer level can be expressed by:

$$\mathbf{u}^F = \sum_{i \in I} E_i (\psi_i u_i|_{\Omega_i}), \quad (21)$$

where ψ_i is a PoU function with compact support within Ω_i and E_i is the extension operator that maps a function defined in Ω_i to a function in Ω , extending by zero outside the subdomains.

Starting from the problem (20), the solution $\mathbf{u}^C$ serves as the initial guess for the first iteration of (17). With the universal definition of $\mathbf{u}^C$, we address the following boundary value problem instead of employing interpolation techniques:

$$\begin{cases} \mathcal{L} \mathbf{u}_i^{F,1} = f & \text{in } \Omega_i, \\ B \mathbf{u}_i^{F,1} = g & \text{on } \partial\Omega \cap \bar{V}_i, \\ \mathbf{u}_i^{F,1} = \mathbf{u}^C & \text{on } \cup_{k'=1}^d (\Gamma_{i,k'}^- \cup \Gamma_{i,k'}^+). \end{cases} \quad (22)$$

A small-scale coarse level correction could be given by the residual problem:

$$\begin{cases} \mathcal{L}\tilde{\mathbf{u}}^{C,\ell} = f - \mathcal{L}\mathbf{u}_i^{F,\ell} & \text{in } \Omega, \\ \mathcal{B}\tilde{\mathbf{u}}^{C,\ell} = g - \mathcal{B}\mathbf{u}_i^{F,\ell} & \text{on } \partial\Omega. \end{cases} \quad (23)$$

This step involves assembling the solution:

$$\mathbf{u}^{F,\ell+\frac{1}{2}} = \tilde{\mathbf{u}}^{C,\ell} + \sum_{i \in I} E_i \left(\psi_i \mathbf{u}_i^{F,\ell} \right). \quad (24)$$

Consequently, the iterative process defined in (17) evolves as follows:

$$\begin{cases} \mathcal{L}\mathbf{u}_i^{F,\ell+1} = f & \text{in } \Omega_i, \\ \mathcal{B}\mathbf{u}_i^{F,\ell+1} = g & \text{on } \partial\Omega \cap \bar{V}_i, \\ \mathbf{u}_i^{F,\ell+1} = \mathbf{u}^{F,\ell+\frac{1}{2}} & \text{on } \cup_{k'=1}^d (\Gamma_{i,k'}^- \cup \Gamma_{i,k'}^+). \end{cases} \quad (25)$$

The iterations persist until the sequence $\mathbf{u}_i^{F,\ell} \rightarrow \mathbf{u}_i^{F,\ell+1}$ converges to $\mathbf{u}_i^{F,\infty}$. Initialization (22) can be viewed as a special coarse level correction if we assume that $\mathbf{u}_i^{F,0} = 0$.

Following the convergence of DDM-2, a few further iterations on the fine mesh only with DDM-1, as defined by (17), yield improved accuracy. We refer to the two-step mixed scheme as “DDM-2+1”.

3. Properties of two-level RFMs

3.1. Reduction in computational cost without sacrificing accuracy

Before introducing DDMs, minimization of the loss function (5) can be reformulated as a linear least-squares problem with sparsity, written in $\min_{\mathbf{u}} \|\Phi \mathbf{u} - \mathbf{f}\|$ for simplicity, where $\Phi \in \mathbb{R}^{N \times M}$. The sparsity induced by PoU naturally encourages us to adopt iterative methods for efficient solutions. However, standard iterative approaches may fall short in accuracy and efficiency due to the poor condition number in RFM, while direct methods, in this case, can lead to significant memory issues and extremely high computational costs [41].

In contrast, minimizing the local loss function (18) results in a dense matrix Φ_i , where $\Phi_i \in \mathbb{R}^{\frac{N}{|I|} \times \frac{M}{|I|}}$, assumed that all $|I|$ sub-problems share the same scale. To ensure accuracy in solving the sub-problems, we employ the direct QR method. Importantly, the matrix Φ_i , which incorporates the conditions from the governing equations and boundary type, remains constant throughout the iterations (17). Consequently, QR decomposition is performed only once and is reused in subsequent iterations. Specifically, the QR method for solving least squares problems consists of two principal steps:

$$\Phi = QR, \quad (26)$$

where Q consists of a series of Householder transformations, and R is an upper triangular matrix. The first decomposition step demands significant computational resources, with a complexity of $\mathcal{O}(NM^2)$ for Φ . When multiple subdomains are decomposed, with a total of $|I|$, the computational cost of local problems is reduced to $\mathcal{O}(NM^2/|I|^3)$ and they can be performed in parallel. This adjustment results in a cumulative computational cost of $\mathcal{O}(NM^2/|I|^2)$, offering a considerable reduction in computational demands.

The second step addresses solving an upper triangular system using the target vector $\mathbf{f}$. Despite multiple executions as referenced in (17), this step incurs a lower computational complexity of $\mathcal{O}(NM + M^2/2)$. Over L iterations, the total computational effort amounts to $\mathcal{O}(L(NM + M^2/2)/|I|)$ during iterations in DDMs.

The combined computational load of these two steps is reduced with domain decomposition as long as the condition $L < M(|I| - 1)/2$ holds. In practice, the number of iterations required for convergence typically falls well below this upper bound, referred to the examples in Section 4. As shown in these examples, introducing DDM can decrease the computational load by a hundredfold or more. With parallel execution, the acceleration effect becomes even more pronounced.

We can establish the stopping criteria by analyzing the least squares residuals. To be more specific, the relative residual is defined as $r = \frac{\|\Phi \mathbf{u} - \mathbf{f}\|}{\|\mathbf{f}\|}$. We could stop the iterations at the fine level when the average change in residuals reaches stability, described by the condition:

$$\frac{1}{|I|} \sum_{i \in I} \left| \frac{r_i^{\ell+1} - r_i^{\ell}}{r_i^{\ell}} \right| < \epsilon, \quad (27)$$

where $|I|$ denotes the cardinality of the set I , reflecting the total number of subdomains.

Consider the case in Section 4.1, where we address the Poisson equation using both the direct method QR and a standard iterative method LSQR with the original RFM. Subsequently, we construct the subdomains aligned with the compact support of PoU functions and decompose the problem into several sub-problems, solving them with DDM-1.

DDM efficiently decomposes the original problem without compromising accuracy, as shown in Table 1. Storing the matrix for the original problem exerts significant memory pressure, despite some alleviation provided by sparse storage techniques. DDM addresses this by densely storing only the non-zero elements of the original problem and leveraging the in-place implementation of the Householder QR, which substantially reduces memory demands.

Table 1
Comparison of L^2 relative errors in different methods.

N	M	Mp	Error(QR)	Error(LSQR) ^a	Error(DDM-1) ^b
1600	1000	1*1*1	5.6171e-02	1.6056e+00	5.6171e-02
10 400	8000	2*2*2	2.6407e-04	1.1917e-02	1.4255e-04
73 600	64 000	4*4*4	6.2582e-06	1.3199e-04	1.2018e-06
550 400	512 000	8*8*8	— ^c	— ^c	3.2526e-07

^a The stopping criterion is a relative error under 1×10^{-8} or reaching twice the matrix's column count in iterations;

^b The stopping criterion follows (27) with $\epsilon = 1 \times 10^{-8}$;

^c Data missing due to insufficient memory.

Table 2
Comparison of error and iteration number for different δ and h .

$\frac{\delta}{2r}$	h	Original Schwarz		Jacobi Schwarz	
		Error	Iterations	Error	Iterations
0.10	2.7500e-02	2.5718e-06	210	1.6239e-06	420
0.25	3.1250e-02	1.2086e-06	89	1.2018e-06	177
0.40	3.5000e-02	1.7308e-06	60	1.5618e-06	116
0.60	4.0000e-02	3.0451e-06	37	2.6438e-06	74

3.2. Convergence and parallelization

Multi-processing programs can access more computing resources via implementations of the Message Passing Interface (MPI). However, the need for communication between subdomains in different processes can introduce inefficiencies. Consider that communication is necessary only between adjacent subdomains. Space-filling curves, including the Hilbert and Z-order curves, can be used to optimize process allocation by trying to cluster adjacent areas within the same process [42].

Besides ensuring an even distribution of subdomains across the processes, it is essential to prevent process blocking to achieve optimal load balancing. While subproblems in the Jacobi-Schwarz method can be processed in parallel without any blocking, this method may exhibit slower convergence compared to the original Schwarz method. We observed that the Jacobi-Schwarz method requires twice as many iterations as the original one, with this requirement being influenced by the overlap size δ . This was theoretically proven in Appendix B with two subdomains. We revisit the case of the Poisson equation in Section 4.1 with multiple subdomain sizes of $4 * 4 * 4$, and applied both the original and the Jacobi-Schwarz method with one level. The results, which confirm the conclusion in the context of multiple subdomains, are provided in Table 2. If the available cores are insufficient for parallel processing of all subdomains, a practical approach is to perform sequential computations within the same process and parallel computations across different processes. Although a larger overlap size δ enhances convergence, it is necessary to balance this against the loss of precision caused by sparser sampling.

It is worth mentioning that only the weights of the output layer need to be communicated. The fixed inner parameters can be generated using the same pseudo-random number seed. In this way, RFM offers a method of compression in representing solutions compared to point clouds or meshes.

3.3. Scalability

A proper initialization can accelerate convergence and enhance stability. In the absence of prior information, using zero as the initial guess is a practical option. In the two-level Schwarz method, incorporating a coarse level introduces a negligible extra cost, especially when numerous subdomains are involved. Yet, this addition significantly enhances both convergence and accuracy. Consider the elasticity equation case in Section 4.3 but solve it with both one-level and two-level Jacobi Schwarz method.

Weak scalability requires that the elapsed time remains constant when maintaining a fixed ratio between the problem size and the number of processing units. In each subdomain, the scale of the first QR decomposition step is fixed. If the number of processing units increases proportionally to the number of subdomains, the first step will naturally meet the weak scalability requirement. Solely from the perspective of computation, if $L < (M/2)/|I|$, the computational load of the second iteration step will not exceed that of the first.

Table 3 demonstrates that the iterations do not dominate the overall computational load, despite iterations counts increase with the number of subdomains. The two-level Schwarz method improves the convergence process, as shown in Fig. 3, achieving relatively accurate results with only a few iterations. Additional post-iterations using the one-level Schwarz method, namely “DDM-2+1”, can then be applied to reach optimal accuracy.

Another potential solution is to adjust the size of the coarse level to be proportional to the number of subdomains. In this case, a multilevel approach may be required to ensure the scalability of the coarse level.

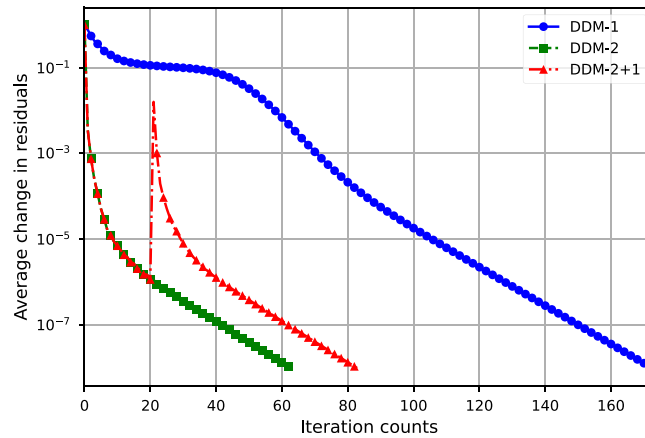


Fig. 3. Convergence comparison of three schemes.

Table 3

Scalability of two-level domain decomposition.

Coarse level		Fine level			DDM-1 ^a		DDM-2 ^b		DDM-2+1 ^c	
N	M	N	M	$ I $	Iter	Error	Iter	Error	Iter	Error
4800	1500	38 400	12 000	2^*2^*2	58	$2.3420\text{e}-06$	20	$2.4769\text{e}-06$	20+1	$2.3908\text{e}-06$
4800	1500	307 200	96 000	4^*4^*4	20	$6.0104\text{e}-02$	–	–	–	–
4800	1500	307 200	96 000	4^*4^*4	173	$6.2379\text{e}-07$	20	$4.2018\text{e}-06$	20+60	$6.2333\text{e}-07$
31200	6000	307 200	96 000	4^*4^*4	173	$6.2379\text{e}-07$	20	$1.5495\text{e}-06$	20+47	$6.2082\text{e}-07$
4800	1500	2 457 600	768 000	8^*8^*8	456	$4.3506\text{e}-07$	20	$1.4220\text{e}-05$	20+131	$4.8892\text{e}-07$

^a DDM-1: one-level Jacobi Schwarz method;^b DDM-2: two-level Jacobi Schwarz method;^c First apply DDM-2, then DDM-1.

4. Numerical examples

In this section, we implement the two-level Jacobi Schwarz method, as outlined in Section 2.2.3, to solve a series of PDEs and evaluate their numerical performance. The overlap factor is fixed at 0.4, and the convergence threshold ϵ is set to 1×10^{-8} . The listed iteration counts represent the total number of iterations, including both the two-level Jacobi Schwarz method and post-iterations using one-level Schwarz. Compared to the results using DDM-2 exclusively, as shown in Appendix C, the results using “DDM-2+1” demonstrate an improvement in accuracy. Up to 64 cores will be used for parallelization, which means that in some cases (the largest ones), multiple subdomains will be allocated to the same process for Gauss–Seidel-type iterations. These problems are solved within a timescale of 100 seconds, a level of efficiency that is quite challenging for traditional methods to achieve while maintaining the same accuracy.

4.1. Poisson equation

Consider the Poisson equation with Dirichlet boundary condition over $\Omega = [0, 1] \times [0, 1] \times [0, 1]$

$$\begin{cases} \Delta u(x, y, z) = f(x, y, z), & (x, y, z) \in \Omega, \\ u(x, y, 0) = g_1(x, y), & u(x, y, 1) = g_2(x, y), \\ u(x, 0, z) = g_3(x, z), & u(x, 1, z) = g_4(x, z), \\ u(0, y, z) = g_5(y, z), & u(1, y, z) = g_6(y, z). \end{cases} \quad (28)$$

We choose an exact solution with mixed frequencies

$$\begin{aligned} u(x, y) = & -\frac{1}{2} \prod_{\ell=1}^3 \left[\frac{3}{2} \cos \left(\pi x_{\ell} + \frac{2\pi}{5} \right) + 2 \cos \left(2\pi x_{\ell} - \frac{\pi}{5} \right) \right] \\ & - \frac{1}{2} \prod_{\ell=1}^3 \left[\frac{3}{2} \cos \left(2\pi x_{\ell} + \frac{4\pi}{5} \right) + 2 \cos \left(4\pi x_{\ell} - \frac{2\pi}{5} \right) \right]. \end{aligned} \quad (29)$$

f and $g_i, i = 1, 2, \dots, 6$ can be explicitly computed according to the exact solution. Detailed results, including derivative errors and computational time (in seconds), are listed in Table 4.

Table 4Relative L^2 error for the Poisson equation.

N	M	$ I $	u error	u_x error	u_y error	u_z error	Iter	Time
1600	1000	1*1*1	4.4261e-02	1.1584e-01	9.4988e-02	9.4297e-02	–	≈ 1
12800	8000	2*2*2	2.5903e-04	9.3385e-04	8.1597e-04	8.6822e-04	17	2.0
102400	64000	4*4*4	1.6949e-06	7.2630e-06	6.6846e-06	7.3262e-06	117	12.9
819200	512000	8*8*8	3.8086e-07	2.8460e-07	2.8040e-07	2.7529e-07	243	80.7

Table 5Relative L^2 error for the homogenization problem.

N	M	$ I $	u error	u_x error	u_y error	u_z error	Iter	Time
1600	1000	1*1*1	2.5982e-01	9.5943e-01	1.0295e+00	9.7251e-01	–	≈ 1
12800	8000	2*2*2	8.9598e-03	3.6661e-02	4.4697e-02	3.5696e-02	98	8.7
102400	64000	4*4*4	5.9023e-05	8.9795e-04	8.9774e-04	7.7663e-04	258	27.1
819200	512000	8*8*8		Reference			582	148.8

Table 6Relative L^2 error of displacements for the linear elasticity problem.

N	M	$ I $	u error	v error	w error	Iter	Time
4800	1500	1*1*1	1.2834e-05	5.0856e-06	7.9407e-06	–	≈ 1
38400	12000	2*2*2	3.5621e-06	1.8283e-06	2.4057e-06	47	9.6
307200	96000	4*4*4	1.1856e-06	6.5105e-07	7.4648e-07	185	30.1
2457600	768000	8*8*8	8.0210e-07	3.0582e-07	8.0604e-07	360	233.6

4.2. Homogenization

We consider the elliptic equation over the $\Omega = [0, 1] \times [0, 1] \times [0, 1]$

$$\begin{cases} -\operatorname{div}(a(\mathbf{x})\nabla u(\mathbf{x})) = f(\mathbf{x}), & \mathbf{x} \in \Omega, \\ u(\mathbf{x}) = 0, & \mathbf{x} \in \partial\Omega, \end{cases} \quad (30)$$

where $a(\mathbf{x}) = e^{h(\mathbf{x})}$, $h(\mathbf{x}) = \sum_{|k| \leq R} (a_k \sin(2\pi \mathbf{k} \cdot \mathbf{x}) + b_k \cos(2\pi \mathbf{k} \cdot \mathbf{x}))$, $\mathbf{k}$ are integer vectors, $R = 2$, and $\{a_k\}$ and $\{b_k\}$ are coefficient with no clear scale separation [43], which are independent, identically distributed random variables with the distribution $\mathbb{U}[-0.3, 0.3]$. The largest case, featuring a total of 819,200 collocation points and 512,000 random features, is used as the reference, and accuracy is recorded in Table 5.

4.3. Linear elasticity

The two-dimensional elasticity problem we consider here is of the following form

$$\begin{cases} -\operatorname{div}(\sigma(\mathbf{u}(\mathbf{x}))) = \mathbf{B}(\mathbf{x}), & \mathbf{x} \in \Omega, \\ \sigma(\mathbf{u}(\mathbf{x})) \cdot \mathbf{n} = \mathbf{N}(\mathbf{x}), & \mathbf{x} \in \Gamma_N, \\ \mathbf{u}(\mathbf{x}) = \mathbf{U}(\mathbf{x}), & \mathbf{x} \in \Gamma_D, \end{cases} \quad (31)$$

where $\sigma : \mathbb{R}^3 \rightarrow \mathbb{R}^3$ is the stress tensor induced by the displacement field $\mathbf{u} : \Omega \rightarrow \mathbb{R}^3$, $\mathbf{B}$ is the body force over Ω , $\mathbf{N}$ is the surface force on Γ_N , $\mathbf{U}$ is the displacement on Γ_D , and $\partial\Omega = \Gamma_N \cup \Gamma_D$.

We consider the Timoshenko beam problem with size $L \times D \times D$. The exact displacement solution for the Timoshenko beam problem is

$$\begin{cases} u = -\frac{P \cdot y}{6EI} \left((6L - 3x)x + (6L - 3z)z + (2 + \nu) \left(y^2 - \frac{D^2}{4} \right) \right), \\ v = \frac{P}{6EI} \left(3\nu y^2(L - x + z) + \frac{(4+5\nu)D^2xz}{4} + (3L - x)x^2 + z^2 \right), \\ w = \frac{P}{6EI} \left(3\nu z^2(L - x + y) + \frac{(4+5\nu)D^2yz}{4} + (3L - z)z^2 \right), \end{cases} \quad (32)$$

where $I = \frac{D^3}{12}$. The Dirichlet boundary condition is imposed on all boundaries based on the exact solution. The material properties are specified as follows: Young's modulus $E = 3150$ Pa, Poisson ratio $\nu = 0.36$. We choose $D = 10$, $L = 10$, and the shear force is $P = 1000$ Pa. Table 6 and Table 7 record the convergence behavior.

5. Engineering applications in structural mechanics

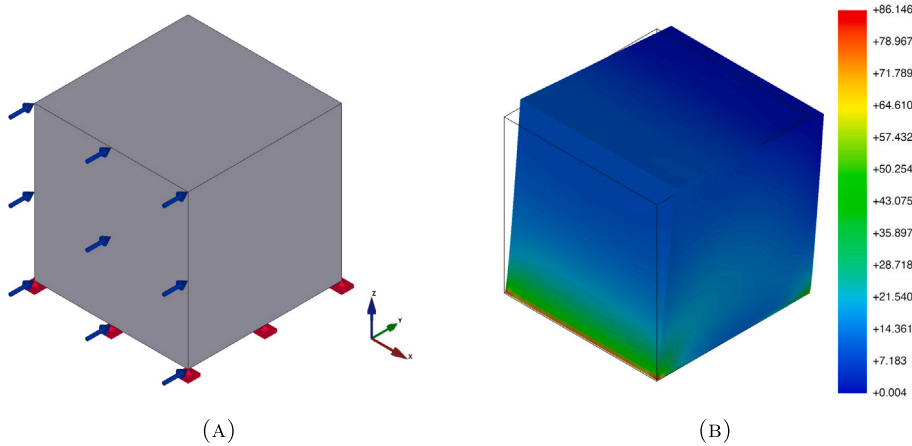
In engineering applications, complex geometries are frequently encountered. In these scenarios, RFM offers a flexible solution by evaluating these geometries through points, from either a basic Cartesian grid or random sampling. Two-level RFMs enhance this

Table 7Relative L^2 error of stresses for the linear elasticity problem.

N	M	$ I $	σ_{xx} error	σ_{yy} error	σ_{zz} error	σ_{xy} error	σ_{xz} error	σ_{yz} error
4800	1500	1*1*1	3.2841e-05	2.2181e-05	2.4076e-05	1.1366e-04	4.1809e-04	4.5539e-05
38 400	12 000	2*2*2	1.3814e-05	1.1169e-05	1.0006e-05	4.9250e-05	1.7309e-04	2.0312e-05
307 200	96 000	4*4*4	6.0448e-06	4.9418e-06	3.9874e-06	2.1990e-05	7.0345e-05	9.0622e-06
2 457 600	768 000	8*8*8	3.9044e-06	3.5715e-06	2.5819e-06	1.5614e-05	4.3407e-05	5.8737e-06

Table 8Comparison of L^2 relative errors of Case I.

Reference	Method	N	M	Disp. error	Stress error
FEM (N = M = 208 425)	RFM	4383	1500	3.0209e-02	1.3900e-01
		37 356	12 000	2.1124e-02	1.0462e-01
		301 782	96 000	8.2015e-03	7.1067e-02
	FEM	9381	9381	4.4630e-03	5.3962e-02
		18 138	18 138	2.8980e-03	4.4407e-02
		54 063	54 063	1.2804e-03	3.0344e-02
RFM (N = 2401 176 M = 768 000)	RFM	4383	1500	3.0311e-02	1.3571e-01
		37 356	12 000	1.7345e-02	9.6562e-02
		301 782	96 000	4.3898e-03	5.3176e-02
	FEM	9381	9381	4.8309e-03	5.3962e-02
		18 138	18 138	3.1632e-03	4.4407e-02
		54 063	54 063	1.4571e-03	3.0344e-02

**Fig. 4.** Case I. (A), Geometry and boundary condition settings; (B), deformation and von Mises Stress (MPa).

methodology to tackle the large-scale system due to increased solution and geometry complexity. This approach is particularly advantageous for geometric surfaces that cannot be precisely represented by polygons or for shapes that cannot be meshed effectively.

In this discussion, we examined various cases in structural mechanics governed by linear elasticity equations. Some of these cases are challenging to solve or even infeasible with traditional FEM. The models are structured using boundary representation (B-rep) in accordance with the STEP standard. Essential geometric algorithms including ray casting, implemented through Open CASCADE Technology (OCCT) [44], are employed in the sampling process.

We will adopt the scheme “DDM-2+1” to complete the following case. The results from RFM with the densest sampling and results from FEM using COMSOL [45] with the solution on the finest mesh as a reference for comparison.

5.1. Case I

Consider a cube with a side length of 10 mm, with its bottom fixed in place. A load with a pressure of 10 MPa is applied to the left side of the cube ($y = 0$), see Fig. 4(a), causing the cube to tilt to the right under the applied force. We compared the results of RFM and FEM in Table 8 and show the deformation computed by RFM in Fig. 4(b) and color it with the magnitude of von Mises stress.

Table 9
Comparison of L^2 relative errors of Case II.

Reference	Method	N	M	Disp. error	Stress error
FEM (N = M = 99 540)	RFM	4032	600	9.3830e-02	7.7471e-01
		24 576	4800	5.2971e-02	6.2679e-01
		193 536	38 400	4.5529e-02	6.0089e-01
	FEM	1845	1845	2.0931e-03	1.1802e-01
		5367	5367	1.3829e-03	9.1979e-02
		19 233	19 233	7.4136e-04	6.2306e-02
RFM (N = 546 048 M = 38 400)	RFM	4032	600	7.4114e-02	7.6568e-01
		24 576	4800	4.1522e-02	2.7592e-01
		193 536	38 400	1.9911e-02	9.5453e-02
	FEM	1845	1845	3.9396e-02	5.8174e-01
		5367	5367	4.0000e-02	5.6368e-01
		19 233	19 233	3.8893e-02	5.6590e-01

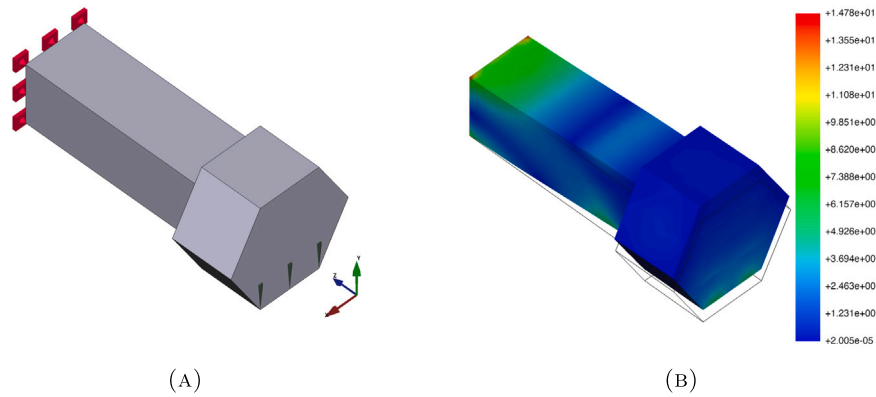


Fig. 5. Case II. (A), Geometry and boundary condition settings; (B), deformation and von Mises Stress (MPa).

5.2. Case II

Consider a bolt comprising a square column and a hexagonal nut. The bolt is fixed on the left side, with traction applied to its nut, resulting in a prescribed displacement of 0.1 mm in the y -direction, see Fig. 5(a). The errors referenced by FEM and RFM are recorded in Table 9 respectively. The deformation and von Mises Stress computed by RFM are shown in Fig. 5(b).

5.3. Case III

Consider replacing the square column in Case II with a cylindrical one while applying the same boundary conditions, see Fig. 6(a). However, the convergence rate from FEM in this case differs significantly. Approximating the surface of the cylinder with polygons introduces geometric errors, leading to poor convergence in the mesh-dependent finite element method, see Table 10. While RFM computations are based on the original geometry, they exhibit better accuracy. We visualize the results of RFM in Fig. 6(b).

5.4. Case IV

We further consider a complex shape stretched from a square, featuring three removed and four filled circular holes. Notably, two of these circles are nearly touching, which presents a significant challenge. This difficulty is evident in the COMSOL software, where mesh generation fails due to the close proximity of these circles. We fix the left side and force the displacement in the y -direction to be 0.1 mm but with displacement in other directions unrestricted, see Fig. 7(a). We evaluate the effectiveness of RFM in complex geometries that mesh-based methods cannot easily handle and show the results in Table 11 and Fig. 7(b).

Computational environment

All numerical experiments in this work were performed on a machine equipped with an AMD EPYC 7763 64-Core Processor and 512 GB of RAM. The implementation was written in C++ and parallelized using OpenMPI across 64 cores. Linear algebra computations were accelerated using OpenBLAS. The code was compiled with GCC 11.3 under Ubuntu 22.04.

Table 10
Comparison of L^2 relative errors of Case III.

Reference	Method	N	M	Disp. error	Stress error
FEM (N = M = 132 681)	RFM	3456	600	9.7206e−02	7.6194e−01
		23 040	4800	4.8977e−02	5.6736e−01
		182 784	38 400	6.2996e−02	4.7374e−01
	FEM	2559	2559	1.4180e−01	3.5209e−01
		6894	6894	1.4327e−01	3.4871e−01
		25 368	25 368	1.4315e−01	3.4631e−01
RFM (N = 521 856 M = 38 400)	RFM	3456	600	8.1138e−02	6.9940e−01
		23 040	4800	6.5194e−02	2.8625e−01
		182 784	38 400	1.8721e−02	9.4584e−02
	FEM	2559	2559	4.3933e−02	4.9833e−01
		6894	6894	4.6509e−02	4.9732e−01
		25 368	25 368	4.6601e−02	4.9587e−01

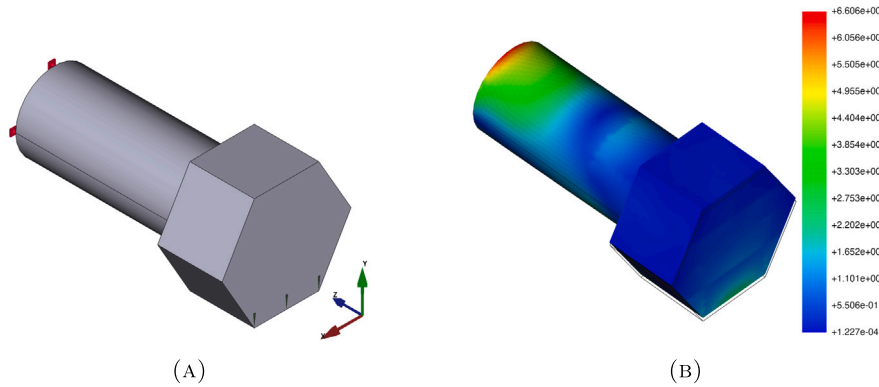


Fig. 6. Case III. (A), Geometry and boundary condition settings; (B), deformation and von Mises Stress (MPa).

Table 11
Comparison of L^2 relative errors of Case IV.

Reference	Method	N	M	Disp. error	Stress error
RFM (N = 544 320 M = 38 400)	RFM	5184	600	1.8230e−01	9.9181e−01
		29 184	4800	7.9233e−02	4.8697e−01
		192 768	38 400	2.2256e−02	1.8581e−01

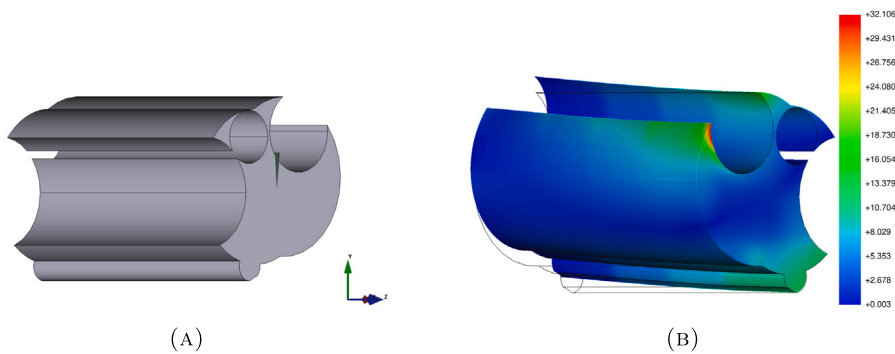


Fig. 7. Case IV. (A), Geometry and boundary condition settings; (B), deformation and von Mises Stress (MPa) (Opposite view).

6. Conclusion

The RFM is a promising approach for solving partial differential equations (PDEs) on complex geometries, effectively overcoming the limitations associated with the mesh dependency of traditional methods. In this work, we strategically incorporate DDM into the RFM, significantly reducing the computational cost of optimization problems. We proposed the two-level RFMs and conducted

multiple numerical experiments to test its properties. Our findings are as follows: (i) The DDM scheme offers accuracy comparable to or exceeding that of the direct method. Additionally, it provides memory advantages by leveraging sparse structures more efficiently than the iterative method. (ii) The two-level DDM-2 scheme further improves scalability with moderate numbers of iterations despite not succeeding in controlling it to a constant. (iii) Scheme “DDM-2+1” is most recommended when aiming for the best accuracy. (iv) The scheme is well-suited for problems with complicated solutions and/or over complex domains and is capable of simulating structures that are challenging to mesh.

CRedit authorship contribution statement

Yifei Sun: Writing – review & editing, Writing – original draft, Software, Methodology. **Jingrun Chen:** Writing – review & editing, Writing – original draft, Methodology, Funding acquisition.

Patent declaration

Research content has been filed for patent application with the patent number 202410745940.7, which is currently under substantive examination.

Declaration of competing interest

The authors declare the following financial interests/personal relationships which may be considered as potential competing interests: Jingrun Chen has patent #202410745940.7 pending to Suzhou Institute for Advanced Research, University of Science and Technology of China. If there are other authors, they declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

The work is supported by NSFC Major Research Plan - Interpretable and General-purpose Next-generation Artificial Intelligence (No. 92370205), NSFC grant 12425113.

Appendix A. Proof of the inequality about L

Consider $N, M, |I|$ are positive integer numbers with $N > M$. Suppose that the inequality $L < M(|I| - 1)/2$ holds, we can derive the following:

$$\begin{aligned}
 L &< M(|I| - 1/|I|)/2 + |I|/2, \\
 L \cdot 2NM &< NM^2|I| - NM^2/|I| + NM|I|, \\
 L \cdot (NM + M^2/2) &< NM^2|I| - NM^2/|I| + NM|I|, \\
 NM^2/|I|^2 + L \cdot (NM + M^2/2)/|I| &< NM^2 + NM, \\
 NM^2/|I|^2 + L \cdot (NM + M^2/2)/|I| &< NM^2 + NM + M^2/2.
 \end{aligned} \tag{33}$$

Appendix B. Convergence analysis of Schwarz method in a simple case of the Poisson problem

Consider the Poisson problem

$$\begin{aligned}
 -\Delta u &= f \text{ in } \Omega, \\
 u &= 0 \text{ on } \partial\Omega.
 \end{aligned} \tag{34}$$

In the 1D case, the sequential Schwarz method can be analyzed independently of the spatial discretization method. Let the domain $\bar{\Omega} = [0, 1]$ be decomposed into two overlapping subdomains $\Omega_1 = [0, \beta]$ and $\Omega_2 = [\alpha, 1]$. The overlap is $\delta = \beta - \alpha$. By linearity of the equation, the error $e_i^n := u_i^n - u|_{\Omega_i}$, $i = 1, 2$, satisfies

$$\begin{aligned}
 -\frac{d^2 e_1^{n+1}}{dx^2} &= f \text{ in } \Omega_1, & -\frac{d^2 e_2^{n+1}}{dx^2} &= f \text{ in } \Omega_2, \\
 e_1^{n+1}(0) &= 0, & e_2^{n+1}(\alpha) &= e_1^{n+1}(\alpha), \\
 e_1^{n+1}(\beta) &= e_2^n(\beta), & e_2^{n+1}(1) &= 0.
 \end{aligned} \tag{35}$$

Thus the errors are linear functions in each subdomain:

$$e_1^{n+1}(x) = e_2^n(\beta) \frac{x}{\beta} \text{ and } e_2^{n+1}(x) = e_1^{n+1}(\alpha) \frac{1-x}{1-\alpha}. \tag{36}$$

And then, we have

$$e_2^{n+1}(\beta) = e_1^{n+1}(\alpha) \frac{1-\beta}{1-\alpha} = e_2^n(\beta) \frac{\alpha}{\beta} \frac{1-\beta}{1-\alpha}. \tag{37}$$

Table 12Relative L^2 error for the Poisson equation if halted at DDM-2.

N	M	$ I $	u error	u_x error	u_y error	u_z error	Iter	Time
1600	1000	1*1*1	4.4261e-02	1.1584e-01	9.4988e-02	9.4297e-02	–	≈ 1
12800	8000	2*2*2	2.7007e-04	9.3051e-04	8.1800e-04	8.7229e-04	9	1.9
102400	64000	4*4*4	5.3366e-05	2.8710e-05	2.7226e-05	2.7897e-05	80	12.1
819200	512000	8*8*8	1.0474e-05	7.3287e-06	6.6224e-06	6.4956e-06	166	68.2

Table 13Relative L^2 error for the homogenization problem if halted at DDM-2.

N	M	$ I $	u error	u_x error	u_y error	u_z error	Iter	Time
1600	1000	1*1*1	2.5982e-01	9.5943e-01	1.0295e+00	9.7251e-01	–	≈ 1
12800	8000	2*2*2	1.3307e-02	3.8341e-02	4.7400e-02	3.7730e-02	52	8.5
102400	64000	4*4*4	3.5736e-04	1.0037e-03	1.0186e-03	8.6888e-04	148	25.2
819200	512000	8*8*8		Reference			503	414.6

Table 14Relative L^2 error of displacements for the linear elasticity problem if halted at DDM-2.

N	M	$ I $	u error	v error	w error	Iter	Time
4800	1500	1*1*1	1.2834e-05	5.0856e-06	7.9407e-06	–	≈ 1
38400	12000	2*2*2	4.2603e-06	1.8886e-06	2.5357e-06	22	8.9
307200	96000	4*4*4	7.5827e-06	3.6483e-06	5.0762e-06	71	26.5
2457600	768000	8*8*8	3.8098e-05	1.6097e-05	2.4620e-05	181	164.1

Table 15Relative L^2 error of stresses for the linear elasticity problem if halted at DDM-2.

N	M	$ I $	σ_{xx} error	σ_{yy} error	σ_{zz} error	σ_{xy} error	σ_{xz} error	σ_{yz} error
4800	1500	1*1*1	3.2841e-05	2.2181e-05	2.4076e-05	1.1366e-04	4.1809e-04	4.5539e-05
38400	12000	2*2*2	1.4651e-05	1.1615e-05	1.0334e-05	5.0458e-05	1.7405e-04	2.0506e-05
307200	96000	4*4*4	1.5665e-05	1.3202e-05	9.9004e-06	4.3942e-05	1.2693e-04	1.4618e-05
2457600	768000	8*8*8	7.9010e-05	5.9463e-05	4.3592e-05	2.5010e-04	6.2365e-04	8.3834e-05

Notice that $\beta = \alpha + \delta$, we have

$$e_2^{n+1}(\beta) = \frac{\alpha}{\alpha + \delta} \frac{1 - \alpha - \delta}{1 - \alpha} e_2^n(\beta) = \frac{1 - \delta/(1 - \alpha)}{1 + \delta/\alpha} e_2^n(\beta). \quad (38)$$

We could define the convergence factor ρ as

$$\rho = \frac{1 - \delta/(1 - \alpha)}{1 + \delta/\alpha}. \quad (39)$$

Is is sufficient to be convergent if $\rho < 1$, that is $\delta > 0$. And the convergence will be faster with larger size of the overlap.

If we talk about the Jacobi Schwarz method that can be executed in parallel in this case, the errors in each subdomain can be written follows:

$$\tilde{e}_1^{n+1}(x) = \tilde{e}_2^n(\beta) \frac{x}{\beta} \text{ and } \tilde{e}_2^{n+1}(x) = \tilde{e}_1^n(\alpha) \frac{1 - x}{1 - \alpha}. \quad (40)$$

It is noticed that $\tilde{e}_1^{2n-1} = e_1^n$, $\tilde{e}_2^{2n} = e_2^n$. Consequently, to match the same error level, the Jacobi Schwarz method will converge as well but with double the number of iterations.

Appendix C. Results if halted at DDM-2

In this section, we revisit the examples discussed in Section 4, but focus on the point where the DDM-2 scheme converges to the threshold. Unfortunately, relying solely on the DDM-2 strategy does not yield optimal accuracy. In some cases, while the precision does not reach the highest level, the overall performance demonstrates convergence in terms of precision, as shown in Table 12 and Table 13. However, in the example of elasticity equations, using only DDM-2 can lead to a decline in accuracy with increased sampling density, as evidenced by Table 14 and Table 15.

Data availability

No data was used for the research described in the article.

References

- [1] Randall J. LeVeque, Finite Difference Methods for Ordinary and Partial Differential Equations: Steady-State and Time-Dependent Problems, SIAM, 2007.
- [2] Olek C. Zienkiewicz, Robert Leroy Taylor, Jian Z. Zhu, The Finite Element Method: Its Basis and Fundamentals, Elsevier, 2005.
- [3] Jie Shen, Tao Tang, Li-Lian Wang, Spectral Methods: Algorithms, Analysis and Applications, vol. 41, Springer Science & Business Media, 2011.
- [4] Xiangrong Li, John S. Lowengrub, Andreas Rätz, Axel Voigt, Solving PDEs in complex geometries: a diffuse domain approach, *Commun. Math. Sci.* 7 (1) (2009) 81–107.
- [5] Fei Yu, Zhenlin Guo, John Lowengrub, Higher-order accurate diffuse-domain methods for partial differential equations with Dirichlet boundary conditions in complex, evolving geometries, *J. Comput. Phys.* 406 (2020) 109174.
- [6] Long Chen, Huayi Wei, Min Wen, An interface-fitted mesh generator and virtual element methods for elliptic interface problems, *J. Comput. Phys.* 334 (2017) 327–348.
- [7] Peter K. Galenko, Dmitri V. Alexandrov, Ekaterina A. Titova, The boundary integral theory for slow and rapid curved solid/liquid interfaces propagating into binary systems, *Phil. Trans. R. Soc. A* 376 (2113) (2018) 20170218.
- [8] Michael Feischl, Thomas Führer, Norbert Heuer, Michael Karkulik, Dirk Praetorius, Adaptive boundary element methods: a posteriori error estimators, adaptivity, convergence, and implementation, *Arch. Comput. Methods Eng.* 22 (3) (2015) 309–389.
- [9] Julien Kockelkoren, Herbert Levine, Wouter-Jan Rappel, Computational approach for modeling intra- and extracellular dynamics, *Phys. Rev. E* 68 (3) (2003) 037702.
- [10] Alfonso Bueno-Orovio, Víctor M. Pérez-García, Spectral smoothed boundary methods: The role of external boundary conditions, *Numer. Methods Partial. Differ. Equ.* 22 (2) (2006) 435–448.
- [11] Travis Askham, Antoine J. Cerfon, An adaptive fast multipole accelerated Poisson solver for complex geometries, *J. Comput. Phys.* 344 (2017) 1–22.
- [12] Alfonso Bueno-Orovio, Víctor M. Pérez-García, Flavio H. Fenton, Spectral methods for partial differential equations in irregular domains: the spectral smoothed boundary method, *SIAM J. Sci. Comput.* 28 (3) (2006) 886–900.
- [13] Weinan E, Jiequn Han, Arnulf Jentzen, Deep learning-based numerical methods for high-dimensional parabolic partial differential equations and backward stochastic differential equations, *Commun. Math. Stat.* 5 (4) (2017) 349–380.
- [14] Jiequn Han, Arnulf Jentzen, Weinan E, Solving high-dimensional partial differential equations using deep learning, *Proc. Natl. Acad. Sci. USA* 115 (34) (2018) 8505–8510.
- [15] Weinan E, Bing Yu, The deep ritz method: A deep learning-based numerical algorithm for solving variational problems, *Commun. Math. Stat.* 6 (1) (2018) 1–12.
- [16] Justin A. Sirignano, Konstantinos Spiliopoulos, DGM: A deep learning algorithm for solving partial differential equations, *J. Comput. Phys.* 375 (2018) 1339–1364.
- [17] Maziar Raissi, Paris Perdikaris, George Em Karniadakis, Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations, *J. Comput. Phys.* 378 (2019) 686–707.
- [18] Yaohua Zang, Gang Bao, Xiaojing Ye, Haomin Zhou, Weak adversarial networks for high-dimensional partial differential equations, *J. Comput. Phys.* 411 (2020) 109409.
- [19] Yunlei Yang, Muzhou Hou, Jianshu Luo, A novel improved extreme learning machine algorithm in solving ordinary differential equations by Legendre neural network methods, *Adv. Difference Equ.* 2018 (1) (2018) 1–24.
- [20] Francesco Calabrò, Gianluca Fabiani, Constantinos Sietos, Extreme learning machine collocation for the numerical solution of elliptic PDEs with sharp gradients, *Comput. Methods Appl. Mech. Engrg.* 387 (1) (2021) 114–188.
- [21] Suchuan Dong, Zongwei Li, Local extreme learning machines and domain decomposition for solving linear and nonlinear partial differential equations, *Comput. Methods Appl. Mech. Engrg.* 387 (1) (2021) 114–129.
- [22] Jingrun Chen, Xurong Chi, Weinan E, Zhouwang Yang, Bridging traditional and machine learning-based algorithms for solving pdes: The random feature method, *J. Mach. Learn.* 1 (3) (2022) 268–298.
- [23] Xurong Chi, Jingrun Chen, Zhouwang Yang, The random feature method for solving interface problems, *Comput. Methods Appl. Mech. Engrg.* 420 (2024) 116719.
- [24] Jingrun Chen, Weinan E, Yixin Luo, The random feature method for time-dependent problems, *East Asian J. Appl. Math.* 13 (3) (2023) 435–463.
- [25] Jiamin Jiang, Pavel Tomlin, Hamdi Tchepeli, Accelerated nonlinear domain decomposition solver for multi-phase flow and transport in porous media, *J. Comput. Phys.* 490 (2023) 112328.
- [26] Haijian Yang, Shuyu Sun, Yiteng Li, Chao Yang, A scalable fully implicit framework for reservoir simulation on parallel computers, *Comput. Methods Appl. Mech. Engrg.* 330 (2018) 334–350.
- [27] Øystein Klemetsdal, Arthur Moncorgé, Olav Moyner, Knut-Andreas Lie, Additive Schwarz preconditioned exact Newton method as a nonlinear preconditioner for multiphase porous media flow, in: *ECMOR XVII, 2020, European Association of Geoscientists & Engineers, 2020*, pp. 1–20.
- [28] Victorita Dolean, Martin J. Gander, Walid Kheriji, Felix Kwok, Roland Masson, Nonlinear preconditioning: How to use a nonlinear Schwarz method to precondition Newton's method, *SIAM J. Sci. Comput.* 38 (6) (2016) A3357–A3380.
- [29] Victorita Dolean, Pierre Jolivet, Frédéric Nataf, An introduction to domain decomposition methods, in: *Other Titles in Applied Mathematics, Society for Industrial and Applied Mathematics, 2015*.
- [30] Feng-Nan Hwang, Xiao-Chuan Cai, A parallel nonlinear additive Schwarz preconditioned inexact Newton algorithm for incompressible Navier–Stokes equations, *J. Comput. Phys.* 204 (2) (2005) 666–691.
- [31] Xiao-Chuan Cai, David E. Keyes, Nonlinearly preconditioned inexact Newton algorithms, *SIAM J. Sci. Comput.* 24 (1) (2002) 183–200.
- [32] Xiao-Chuan Cai, William D. Gropp, David E. Keyes, Robin G. Melvin, David P. Young, Parallel Newton-Krylov-Schwarz Algorithms for the Transonic Full Potential Equation, *Tech. Report NAS 1.26:198341*, 1996.
- [33] Satish Balay, William Gropp, Lois Curfman McInnes, Barry F. Smith, PETSc, the portable, extensible toolkit for scientific computation, *Argonne Natl. Lab.* 2 (17) (1998).
- [34] Frédéric Hecht, New development in freefem++, *J. Numer. Math.* 20 (2012).
- [35] Ansys Inc, Corp. Philanthr. Rpt. 38 (2) (2023) 4.
- [36] Michael Smith, ABAQUS/standard user's manual, version 6.9, 2009.
- [37] Qi Sun, Xuejun Xu, Haotian Yi, Domain decomposition learning methods for solving elliptic problems, *SIAM J. Sci. Comput.* 46 (4) (2024) A2445–A2474.
- [38] Ben Moseley, Andrew Markham, Tarje Nissen-Meyer, Finite basis physics-informed neural networks (FBPINNs): A scalable domain decomposition approach for solving differential equations, *Adv. Comput. Math.* 49 (4) (2023) 62.
- [39] Victorita Dolean, Alexander Heinlein, Siddhartha Mishra, Ben Moseley, Multilevel domain decomposition-based architectures for physics-informed neural networks, *Comput. Methods Appl. Mech. Engrg.* 429 (2024) 117116.
- [40] Hermann Amandus Schwarz, Über einen Grenzübergang durch alternierendes Verfahren, *Vierteljahr. Naturforschenden Ges. Zürich* 15 (1870) 272–286.
- [41] Jingrun Chen, Weinan E, Yifei Sun, Optimization of random feature method in the high-precision regime, *Com. Appl. Math. Comput.* (2024).
- [42] Hans Sagan, Space-filling curves, in: *Universitext, Springer, New York, NY, 1994*.
- [43] Houman Owhadi, Lei Zhang, Metric-based upscaling, *Commun. Pur. Appl. Math.* 60 (5) (2007) 675–723.
- [44] Open CASCADE technology (OCCT), 2024, Available at: <https://www.opencascade.com/>.
- [45] COMSOL AB, COMSOL multiphysics® v. 6.2, 2024, <https://www.comsol.com>.